WEEK 7 -- SQL: DDL/DML and Database Design

Setup: *Use the robotics practice database. Create all new objects in week07_lab. Each task has independent table names. Use 2026-10-01 as today. Run one task inside BEGIN and ROLLBACK in the same connection; HW7.9 requires an explicit transaction.*

1. In-Class Exercises

1.1. IC7.1 -- A robot registry with reliable defaults

Create ic71_robots with a generated integer id, unique required serial number up to 24 characters, required model name, commissioning date, active flag defaulting to true, and positive maximum_payload_kg. Calibration notes may be NULL. Insert two robots and prove that NULL payload and zero payload are rejected. Explain why a CHECK alone does not reject NULL.

1.2. IC7.2 -- Operator certifications as a bridge

Design ic72_operators, ic72_certifications, and ic72_operator_certifications. An operator can hold many certifications, but each operator-certification pair is unique and has a required awarded_on date. Insert two operators and two certifications, then three valid assignments. Safely demonstrate a repeated pair and an unknown certification failing. Return every operator with their certification count, including zero.

1.3. IC7.3 -- Unique active operator contacts

Create ic73_contacts with id, operator_name, email, and is_active. Among active rows only, require uniqueness of lower(btrim(email)); trimming here removes ASCII spaces. Ignore NULL and blank emails in this rule. Insert an active contact and an inactive contact sharing the normalized email. Prove that activating the duplicate fails while multiple active missing emails are allowed.

1.4. IC7.4 -- Maintenance-window constraints

Create ic74_maintenance_windows with a primary key and required start_at/end_at timestamps. Duration may be zero but must not exceed 14 days. An optional inspection_at must fall inside the window, including its endpoints. Insert zero-duration and exactly-14-day windows. Safely prove that a reversed window, a 14-day-plus-one-second window, and an outside inspection time are rejected.

1.5. IC7.5 -- Retiring a robot under four delete rules

Create ic75_robots and four child tables, one each for NO ACTION, RESTRICT, CASCADE, and SET NULL. Give each rule its own robot and one dependent mission record. Delete the robots, catching the two expected restrictions. Show surviving child counts and non-NULL robot references. Explain which rule can use a deferred foreign-key check and why SET NULL needs a nullable column.

1.6. IC7.6 -- Repeatable operator onboarding

Create ic76_onboarding with operator_id referencing public.operators, a required normalized email snapshot, and created_at. Allow one onboarding record per operator. Insert records for undeleted operators whose email is not NULL or blank after trimming ASCII spaces. Run the insert twice, using a missing-row test plus a conflict guard. Prove that the second run adds no records.

1.7. IC7.7 -- Refresh operator mission-cost summaries

Create ic77_operator_costs containing every operator id and a required nonnegative completed_cost, initially 999 PLN. Use one UPDATE with aggregated public mission data to replace all values with each operator's all-time completed mission cost. Operators without completed missions must become zero. Guest missions belong to nobody's summary. Verify every result with a separate correlated aggregate.

1.8. IC7.8 -- A deterministic cleanup preview

Copy operator ids and lifetime_cost into ic78_operator_costs, without foreign keys. Preview the bottom floor(10% of row count) operators by lifetime_cost ascending, breaking ties by id. Then delete exactly those rows with a subquery and show their ids with RETURNING. Verify the removed count and remaining count. Work only on the lab copy.

1.9. IC7.9 -- Robot parts with physical variants

Create ic79_parts and ic79_variants. A part can have variants distinguished by voltage and connector; each combination is unique within its part. Require a globally unique SKU and nonnegative integer stock. Insert two parts and three variants, then prove negative stock and a repeated combination fail. Return total stock for every part, including parts with no variants.

1.10. IC7.10 -- A normalized mission-parts bill

Design ic710_parts, ic710_missions, and ic710_mission_parts. Each mission may consume many parts; each mission-part pair is unique. Preserve historical unit_cost and require positive quantity. Add one mission using two motors at 40 PLN each and three sensors at 10 PLN each. Report its total cost and explain why the part's current list price must not rewrite historical mission costs.

2. Homework Exercises

2.1. HW7.1 -- Migrate a flat deployment extract

Create hw71_flat with three rows: mission 1/operator 10/model 100/cost 40, mission 1/operator 10/model 101/cost 60, and mission 2/operator 20/model 100/cost 50. Normalize into hw71_operators, hw71_missions, and hw71_lines, preserving both mission totals. Add appropriate keys and references. Verify two operators, two missions, three lines, and a total cost of 150 PLN.

2.2. HW7.2 -- Soft-deleted robot registrations

Create hw72_robots with three rows: active, deleted on 2026-07-01, and deleted on 2026-09-01. Expose active rows through hw72_active_robots. Soft-delete the active robot at the fixed reporting instant, then hard-delete registrations deleted strictly more than 90 days earlier. Show the surviving rows and explain why the view becomes empty although two stored records remain.

2.3. HW7.3 -- Audit robot configuration changes

Create hw73_configs(robot_id, speed_limit) and hw73_history. A trigger must copy the old speed limit only when its value actually changes; a no-op update creates no history. Keep history independent of the current robot row. Insert robot 1 at speed 10, repeat 10, then change to 15 and 20. Verify two history values, 10 then 15, and current value 20.

2.4. HW7.4 -- Version-aware telemetry UPSERT

Create hw74_metrics keyed by robot_id and metric_day, with value and source_updated_at. Insert robot 1/day 2026-09-30/value 10/version 10:00. Apply value 20/version 11:00, then replay value 99/version 09:00. Update an existing metric only when the incoming version is strictly newer. Verify that 20 survives and an identical replay changes zero rows.

2.5. HW7.5 -- A robot-family hierarchy

Create hw75_families with id, name, and an optional parent reference. Reject direct self-parenting. Insert Autonomous Systems, its child Mobile Robots, and grandchild Inspection Rovers, then display each family with its parent's name. Demonstrate that self-parenting fails. Explain why this CHECK does not prevent longer cycles and describe a suitable additional safeguard, without implementing it.

2.6. HW7.6 -- Rename a field behind compatibility views

Create hw76_robots(id, full_name) and an old-client view hw76_old_robots exposing those columns. Rename the base column to name and create hw76_new_robots exposing id/name. Show that both views read the same row, then update through the old view. Explain a deployment sequence that moves old readers to the compatibility view before the rename and retires it after migration.

2.7. HW7.7 -- Monthly robot-event partitions

Create hw77_events partitioned by event_at with generated event_id, robot_id, and event_type. Use primary key(event_at, event_id), with September and October partitions and no default partition. Insert one event just before October and one exactly at October's start. Prove that an August event fails. Explain why event_id alone cannot be a declared unique key on this parent.

2.8. HW7.8 -- Generated robot access codes

Create hw78_robots with a generated id, required name, and a default eight-character hexadecimal access_code. Enforce uniqueness and exact hexadecimal format. Insert five robots without supplying codes and show the generated values. Prove that a duplicate explicit code fails. Explain why the UNIQUE constraint, rather than randomness alone, guarantees stored uniqueness and why an application may need to retry a collision.

2.9. HW7.9 -- Loading children before their parents

Create hw79_manufacturers and hw79_models with a DEFERRABLE INITIALLY DEFERRED foreign key. Within one transaction, insert model 100 referencing manufacturer 1 before inserting manufacturer 1. Force constraint checking after both inserts. Then prove that a model referencing missing manufacturer 99 fails when constraints are immediate. Explain how this differs from disabling integrity checks during a load.

2.10. HW7.10 -- Indexes for two robotics workloads

Copy public.missions into hw710_missions. Design one partial index for an operator's latest completed missions, ordered by started_at and id descending, and another covering index for completed costs within a start-time range. Create both, run ANALYZE, and show their definitions. Justify key order, predicates, and included columns; explain why this small fixture may still use sequential scans.